# BeyondGorgeous

Design & Roadmap

Version 0.4.0 · Architecture, scope, roadmap, risks & sign-off plan

**BeyondGorgeous**

**Version 2 — Architecture Design & Delivery Roadmap**

E-commerce platform design document

| Field | Detail |
|---|---|
| **Project** | **BeyondGorgeous beauty e-commerce platform** |
| Document | Architecture Design & Delivery Roadmap |
| Version | 0.4.0 (aligned with ARCHITECTURE.md) |
| Date | 25 June 2026 |
| Status | Draft for review |
| Document owner | Ashish Jain (Founder / Product Owner) |
| Prepared with | Claude (build & design partner) |
| Primary domain | beyondgorgeous.in |
| Source repository | github.com/ajainx3/beyondgorgeous |
| Canonical source | docs/ARCHITECTURE.md (version-controlled) |

*Confidential — internal planning document.*

**Table of contents**

## 1. Executive summary

BeyondGorgeous is a private-label beauty brand. Version 1 delivered a live, responsive showcase website at beyondgorgeous.in, hosted on Cloudflare, with a Nykaa-inspired design, eight product categories, sample products, an embedded brand video, and a “Launching Soon” watermark. The domain, DNS, SSL, and alias redirects (.online) are all live and stable.

Version 2 converts that showcase into a real, transactional store. The defining characteristic of the business is a hybrid supply-and-fulfillment model: the brand, storefront, pricing, and content are owned by BeyondGorgeous, while production, labelling, packing, and shipping are handled by multiple whitelisted vendors (e.g. BO International) for dropship products, and by BeyondGorgeous directly for products it stocks and self-fulfils.

This document defines the target architecture and the phased delivery roadmap to reach a sellable store. It is written for two audiences: the founder (as a plain-language plan and decision record) and the build partner (as the technical blueprint). The technical detail is mirrored in the version-controlled file docs/ARCHITECTURE.md; this Word document adds the project-management layer — scope, milestones, dependencies, risks, and success criteria.

**Headline plan:** five sequential phases — (1) Catalog & admin with manual ingestion, (2) Customer accounts & cart, (3) Checkout & payments, (4) Fulfillment & vendor automation, (5) AI customer support — building on a vendor- and fulfillment-ready foundation so that no rework is required as automation is layered in.

# 2. Objectives & success criteria

## 2.1 Business objectives

- Launch a real, sellable beauty store under the BeyondGorgeous brand.
- Manage a product catalog with full control over categories, products, variants, images, and pricing.
- Automate catalog data (stock, price, availability, new items) from multiple vendor sources, minimising manual effort.
- Support both dropship and self-fulfilled shipping, selectable per product.
- Be discoverable on Google for product and keyword searches.
- Offer responsive, AI-assisted customer support with human escalation.

## 2.2 Success criteria (definition of done for V2)

| Area | Success criterion | Measure |
|---|---|---|
| **Catalog** | **Founder can add/edit/remove products and variants without developer help** | **Admin panel in daily use** |
| Ingestion | A vendor price/stock sheet updates the catalog without manual re-keying | Spreadsheet upload → live update |
| Commerce | A customer can register, add to cart, pay, and receive confirmation | End-to-end test order completes |
| Fulfillment | Orders route to the correct vendor or warehouse automatically | Routing verified for both models |
| SEO | Product pages are server-rendered with structured data and indexed | Pages appear in Search Console |
| Support | Common questions answered by AI; complex issues raise a Jira ticket | Escalation creates ticket with context |

## 2.3 Key performance indicators (post-launch)

- Catalog maintenance effort: manual hours per 100 catalog updates (target: trending down as automation lands).
- Organic traffic: indexed pages and impressions in Google Search Console.
- Conversion: completed orders ÷ sessions.
- Support deflection: share of queries resolved by AI without a ticket.
- Operational reliability: ingestion success rate per source; order routing accuracy.

# 3. Scope

## 3.1 In scope (Version 2)

| Capability | Notes |
|---|---|
| **Product catalog & admin panel** | **Categories, brands, products, variants, images, attributes; password-protected admin** |
| Multi-source catalog ingestion | Manual entry and spreadsheet/CSV in Phase 1; API, scheduled pull, and email-in in Phase 4 |
| Customer accounts | Guest checkout, phone-OTP login, profiles, addresses (pincode), order history |
| Cart & checkout | Cart, wishlist, back-in-stock, atomic stock reservation |
| Payments | Razorpay (UPI, cards, net banking, wallets) and cash-on-delivery |
| Orders, returns & refunds | Order state machine, cancellations, RMA / reverse logistics, refunds |
| Tax & invoicing | GST-compliant invoices (HSN) and credit notes |
| Promotions | Coupons and automatic discount rules |
| Notifications | Email, SMS, and WhatsApp transactional messaging |
| Hybrid fulfillment | Dropship and self-fulfilled; serviceability/COD; order routing |
| Inventory & replenishment (ERP) | Stock ledger + batch/expiry; safety stock; auto-reorder/alerts |
| Content & reviews | CMS-lite banners/collections/blog; ratings & reviews |
| Engineering foundations | Environments, CI/CD, migrations, tests, observability, security (Phase 0) |
| Compliance | GST, DPDP, e-commerce rules + Grievance Officer, cosmetics labeling |
| Marketplace readiness | Seller-aware, listing-based design; build deferred to a future track |
| Shipping integration | Courier aggregator for self-fulfilled; tracking ingestion for dropship |
| SEO foundation | Server-side rendering, clean URLs, structured data, sitemap |

| Capability | Notes |
|---|---|
| AI customer support | Chatbot grounded in catalog + orders + FAQ; Jira escalation |

### 3.2 Out of scope (deferred beyond V2)

- Native mobile app (the beyondgorgeous.app domain is reserved for this future track).
- Multi-currency / international shipping (launch is India-first, INR).
- Building the third-party-seller marketplace (seller portal, commissions, payouts). The architecture is designed to support it — single-seller at launch, marketplace-ready — but the build is a deferred future track.
- Loyalty programmes, subscriptions, and advanced promotions engine (candidate for V3).
- Warehouse management system (WMS); inventory is tracked at catalog level, not bin/shelf level.
- Advanced analytics / BI dashboards beyond standard Cloudflare and Search Console reporting.

## 4. Business model & context

BeyondGorgeous operates a private-label, hybrid-fulfilment model. It is not a traditional inventory-holding retailer, nor an open marketplace. The brand sits on the front; supply and fulfilment sit behind it in two coexisting modes.

| Dimension | Owned by BeyondGorgeous | Owned by vendors |
|---|---|---|
| **Brand & storefront** | **Yes** | **No** |
| Marketing content & images | Yes | No |
| Selling price | Yes | No |
| Production, labelling, packing | Self-fulfilled lines only | Dropship lines |
| Stock / availability | Self-fulfilled lines | Dropship lines (synced in) |
| Shipping | Self-fulfilled (via partner) | Dropship (vendor ships) |

Two design consequences follow and shape the entire architecture:

1. Every data field has a single owner, so external data flows in automatically and is never re-keyed by hand.

2. The storefront must never depend on a vendor system being fast or online; vendor data is cached as last-known-good and degrades gracefully.

The platform launches single-seller but is designed seller-aware: a catalog product is separated from a seller's listing (offer), and cart and orders reference the listing. Opening to third-party sellers later is therefore additive, not a redesign (see the Marketplace readiness and roadmap sections).

# 5. Architecture design

## 5.1 Architecture principles

- **Modular monolith.** One application on Cloudflare Workers, cleanly divided into modules. Microservices are deliberately avoided at this scale.
- **Single source of truth.** Each field is owned and edited in exactly one place, with optional manual override locks.
- **One catalog, many sources.** A hub-and-spoke ingestion pipeline normalises every input format into one canonical catalog.
- **Resilience by default.** External calls are timed-out, retried, and queued; the storefront reads from our own database.
- **SEO and security are designed in, not bolted on.** Server-side rendering and least-privilege access are foundational.

## 5.2 Layered overview

| Layer | Responsibility |
|---|---|
| **Actors** | **Customers, founder/admin, vendors, support agents** |
| Edge / UI | Next.js storefront + admin panel, server-rendered on Cloudflare Workers |
| Application core | Catalog, Ingestion, Cart & Orders, Fulfillment, Customers, Support & AI |
| Integration layer | Adapters/connectors: ingestion sources, payments, shipping, AI, Jira |
| Async backbone | Queues (async jobs, retries) and Cron triggers (scheduled sync) |
| Data stores | D1 (SQL), R2 (images + feed files), KV (cache/sessions/flags) |
| External partners | Vendor systems, Razorpay, courier APIs, Jira Cloud, Claude API |

## 5.3 Source of truth & field ownership

This is the principle that removes manual catalog maintenance. A representative mapping:

| Field | Owner | Updated by |
|---|---|---|
| **Name, description, how-to-use** | **BeyondGorgeous** | **Admin panel** |
| Images, marketing tags | BeyondGorgeous | Admin panel |
| Selling price | BeyondGorgeous | Admin (markup over cost) |
| Vendor cost price | Vendor | Ingestion |
| Stock (dropship) | Vendor | Ingestion (auto) |
| Stock (self-fulfilled) | BeyondGorgeous | Warehouse/admin |
| Lead time, weight, dimensions | Vendor / us | Ingestion / admin |
| Discontinued flag | Vendor | Ingestion (auto-hides product) |

## 5.4 Data model (core entities)

Stored in Cloudflare D1 (SQL), grouped by concern. Catalog (the “what”): categories (hierarchical), brands, products (the canonical spec), variants (shade/size SKUs), images, attributes, tags. Marketplace (the “who sells it”): sellers (the house seller seeded as #1) and listings/offers (a seller’s price, stock, and fulfilment on a variant — cart and orders reference listings). Inventory & fulfilment: locations (warehouse and vendor sources), a stock ledger of movements with a derived snapshot (on-hand, reserved, available), reorder policies, and purchase orders. Vendor & ingestion: vendors, ingestion sources, field mappings, ingestion batches, staging items, and vendor-product mappings. Later phases add customers, addresses, carts, orders, shipments, payments, seller payouts, reviews, and support tickets.

## 5.5 Catalog ingestion (multi-source)

BeyondGorgeous acts as its own PIM (Product Information Management) hub. Many source types feed one pipeline:

| Source | How it works | Best for |
|---|---|---|
| **Vendor API** | **Webhooks + scheduled pull via a per-vendor adapter** | **Real-time stock/price** |
| Spreadsheet / CSV | Upload in admin; stored in R2; parsed | Bulk edits, smaller vendors |
| Scheduled file pull | Cron fetches from SFTP / Google Sheet / R2 folder | “Drop a file” vendors |
| Email-in | Dedicated inbox routed to a Worker that parses attachments | Email-only vendors |
| Manual entry | Admin form | New items, one-offs |

**Pipeline:** receive → normalise → validate → map → (review/approve) → publish (idempotent upsert). New items and large changes pause for human approval; routine stock/price updates auto-publish. Field-ownership locks are always respected.

## 5.6 Fulfillment models (hybrid)

| Model | Who ships | Inventory truth | Tracking |
|---|---|---|---|
| **Dropship** | **Vendor** | **Vendor (synced)** | **Ingested from vendor** |
| Self-fulfilled | Us, via courier partner | Our count | From shipping partner |
| Hybrid (same product) | Either source | Sum across locations | Depends on source |

An order-routing engine decides, per line item, where each item is fulfilled — based on fulfillment type, live availability, and (later) cost and destination — and can split a single order across a vendor and our warehouse. Stock is reserved at checkout to prevent overselling. Self-fulfilled lines ship via a courier aggregator (one integration, many couriers); dropship lines use the vendor’s shipping with tracking ingested back to us.

## 5.7 Marketplace readiness

Launch posture is single-seller; the design is seller-aware so opening to third-party sellers later is additive, not a rebuild. The hinge: separate the catalog product from the sellable listing (offer), and have cart and orders reference the listing.

- **Sellers.** BeyondGorgeous is seeded as the house seller; third-party sellers are added as rows later.

- **Listings (offers).** A seller's price, stock, and fulfilment on a variant. At launch: one seller, one listing per variant.
- **Cart & orders reference listings,** so they already carry seller, price, and fulfilment — multi-seller becomes additive.
- **Payments.** Choose a gateway with marketplace split-settlement (Razorpay Route) so future payouts need no re-integration.

**Unblocked for later (not built now):** seller onboarding & portal, multi-offer buy box, commission & payouts, per-seller shipping, ratings, store pages, and GST/tax.

## 5.8 Inventory management & replenishment (ERP)

Designed like an ERP planner, primarily for owned / self-fulfilled stock. On-hand is derived from a stock ledger of movements (receipts, sales, returns, adjustments) rather than a bare counter — giving auditability and the data for replenishment.

Stock states per SKU per location: on-hand, reserved, available (= on-hand − reserved), on-order (open purchase orders), and projected (= available + on-order).

Each SKU carries a reorder policy — safety stock, reorder point, reorder quantity, lead time, preferred vendor (internal or external), and a replenishment mode:

| Mode | Action when available stock ≤ reorder point |
|---|---|
| **auto_po** | **Create and send a Purchase Order to the preferred vendor automatically** |
| approval | Create a draft Purchase Order and alert for human approval |
| alert_only | Raise a low-stock alarm / Jira ticket; no Purchase Order |

A replenishment engine runs on a schedule and on every stock decrement. Purchase Orders have a lifecycle (draft → sent → confirmed → received); sending one reuses the integration layer outbound (API or email to the vendor), mirroring ingestion. Goods receipt posts ledger entries. Low-stock, out-of-stock, overstock, and overdue-PO conditions raise admin alarms and optional Jira tickets. Safety stock is fixed per SKU at launch and can become statistical later using sales velocity from the ledger.

**Cosmetics specifics:** inventory tracks batch/lot with manufacturing and expiry dates and allocates by FEFO (first-expiry-first-out), enabling expiry alerts and batch recalls. Vendor scorecards (lead-time adherence, defect/RTO rate) inform preferred-vendor selection; landed cost feeds margin, with ABC/XYZ and demand forecasting as later additions.

## 5.9 SEO architecture

SEO is a rendering decision, not a late add-on. All public pages are server-side rendered so Google receives complete HTML. URLs are clean and keyword-rich; slugs are stable. Each page

emits a unique title, meta description, and canonical URL. Structured data (JSON-LD) for Product, BreadcrumbList, Organization, and WebSite drives rich results — price, star ratings, and breadcrumbs — in Google. An auto-generated XML sitemap and robots.txt support crawling, and the site is registered in Google Search Console. Action item carried into Phase 1: the current category page is a client component and must be rebuilt server-side for SEO.

### 5.10 AI customer support & Jira

A Claude-powered chatbot answers from three grounded sources: the product catalog, the signed-in customer's own order data, and a curated FAQ. When it cannot resolve an issue, it automatically creates a Jira ticket containing the conversation transcript and order context, and returns a ticket reference to the customer. The bot never performs irreversible actions (refunds, cancellations) itself; those always escalate to a human.

### 5.11 API surface

| Group | Caller | Auth | Examples |
|---|---|---|---|
| **Storefront** | **Website** | **Public / session** | **products, cart, checkout, chat** |
| Admin | Founder | Cloudflare Access | manage catalog, vendors, orders |
| Ingestion | Vendor systems / feeds | API key + signed | stock/price webhooks, uploads |
| Webhook receivers | Razorpay, couriers | Signature verified | payment, shipment updates |
| Internal jobs | Cron + Queues | Internal | feed pulls, routing, emails |

## 5.12 Technology stack

| Need | Service |
|---|---|
| **Hosting / compute** | **Cloudflare Workers** |
| Framework | Next.js (App Router) |
| Database | Cloudflare D1 (SQL) |
| File / image storage | Cloudflare R2 |
| Async jobs / scheduling | Cloudflare Queues + Cron Triggers |
| Email ingestion | Cloudflare Email Routing → Worker |
| Cache / sessions | Cloudflare KV |
| Admin authentication | Cloudflare Access |
| AI chatbot | Claude API (latest model) |
| Search | Dedicated index (Cloudflare AI Search / external) |
| File optimisation | Cloudflare Images |
| Customer auth | Phone-OTP provider + KV sessions |
| Messaging | Email + SMS + WhatsApp providers |
| Payments | Razorpay + Route + COD (proposed) |
| Shipping (self-fulfilled) | Courier aggregator, e.g. Shiprocket (proposed) |
| Bot / security | Turnstile, WAF, rate limiting |
| Observability | Logs/metrics + error tracking |
| Support tickets | Jira Cloud |

## 5.13 Customers, accounts & checkout

Guest checkout is first-class — no forced registration. Primary login is phone number + OTP (India-standard), with email/social optional; Turnstile guards auth and checkout. Indian-format addresses with pincode; carts merge from guest to account on login; back-in-stock subscriptions tie to inventory.

## 5.14 Orders, returns & refunds

Orders follow a state machine (created → payment_pending → confirmed → processing → shipped → delivered → closed, plus cancelled / returned / refunded / RTO) and support partial shipments and per-seller splits. Cancellations (pre-dispatch) and returns/RMA (post-delivery) carry reasons, windows and approval, with reverse pickup and refunds to the original method or store credit. Returned stock re-enters the ledger after QC and batch/expiry check. Every transition emits an event.

## 5.15 Payments (including COD)

Razorpay (UPI, cards, net-banking, wallets), Route-capable for future seller payouts, plus cash-on-delivery gated by pincode serviceability with RTO reconciliation. PCI scope stays with Razorpay and card data is tokenized per RBI rules (never stored). Idempotency keys and signed webhooks protect payment/order creation; reconciliation and failed-payment / abandoned-cart recovery run as jobs.

## 5.16 Promotions & pricing

A coupon engine (e.g. GORGEOUS20) supports percent / amount / free-shipping discounts with scope and constraints (minimum cart, usage limits, validity, first-order), automatic rules (e.g. buy-2-get-1), and margin guardrails using landed cost. Applied discounts are recorded per order for accounting.

## 5.17 Notifications & messaging

A provider-agnostic notifications service covers email, SMS, and WhatsApp (Business API — important in India), driven by domain events through the outbox for guaranteed delivery: OTP, order placed / shipped / delivered, refunds, back-in-stock, and abandoned cart.

## 5.18 Tax & invoicing (GST)

GST is computed per line from HSN codes and state-based rates, producing compliant tax invoices (stored in R2) and credit notes for returns, with e-invoicing once thresholds are crossed. Tax is a separate module so rules can change without touching catalog or orders.

## 5.19 Content, merchandising & reviews

CMS-lite manages banners, collections, featured lists and a blog (how-to / ingredient content — the leading organic-traffic driver for beauty) instead of hardcoding. Merchandising adds bundles, cross-sell / upsell and later personalised recommendations. Reviews / UGC (ratings, text, photos, verified badge, moderation) drive conversion and feed Review structured data.

## 5.20 Data & platform architecture

Decisions that protect scalability: a data-access (repository) layer so D1 can be replaced or augmented without rewriting logic; a dedicated search index for product search/facets (not LIKE queries on D1); a separate analytics/event sink (high-volume events never hit D1); an event/outbox pattern so side-effects are never lost; API-first and versioned (/v1) so the future mobile app reuses the same APIs; money as integer minor units; and idempotency keys on all commerce mutations.

**Scalability cap noted:** Cloudflare D1 (SQLite) has single-primary writes and size/concurrency limits — fine for catalog and orders, but search and analytics are deliberately kept off it, and the data-access layer keeps a migration path open.

## 5.21 Engineering foundations & delivery

Established up front (Phase 0), non-negotiable before real money flows: separate dev / staging / production environments; CI/CD with preview deployments (no direct-to-prod from a laptop); versioned D1 schema migrations through CI; unit + Playwright end-to-end tests with a QA gate per phase; observability (logs, metrics, error tracking, SLOs, alerts); backups & DR (D1 exports, R2 versioning); security hardening (WAF, rate limiting, Turnstile, secret / dependency scanning); Cloudflare Images and performance budgets; and a reusable component / design system with accessibility (WCAG).

## 5.22 Compliance & legal (India)

A first-class workstream running in parallel from Phase 0: Consumer Protection (E-Commerce) Rules 2020 (published return/refund/shipping policies and a mandatory Grievance Officer); GST (registration, HSN-based invoices, credit notes, e-invoicing at threshold); DPDP Act 2023 (consent including cookies, privacy policy, data-principal rights, retention limits); cosmetics regulation (Drugs & Cosmetics Act / CDSCO labeling, import licensing if imported, expiry / batch handling); and payments (PCI via Razorpay, RBI tokenization). Seller KYC and per-seller GST apply when the marketplace opens.

## 6. Non-functional requirements

| Concern | Approach |
|---|---|
| **Security** | **Least-privilege auth per API group; admin behind Cloudflare Access; signed vendor webhooks; secrets in Workers secret store; card data handled by Razorpay, never on our servers** |
| Resilience | Storefront reads last-known-good from our DB; external calls timed-out, retried, circuit-broken; queues absorb spikes and failures |
| Idempotency | All ingestion and webhook handlers are safe to receive duplicates |
| Data quality | Validation gates and human approval for new items prevent bad data reaching customers |
| Performance | Cloudflare edge delivery; optimised, lazy-loaded images; strong Core Web Vitals |
| Observability | Structured logs; per-source ingestion metrics; alerts on repeated failures, payment errors, low stock |
| Privacy | Minimal customer data; secure handling of addresses and order history; clear policies |
| Compliance | GST invoices; DPDP consent; e-commerce rules + Grievance Officer; cosmetics labeling; RBI tokenization |

## 7. Delivery roadmap

Foundations first (Phase 0), then commerce, then automation, plus two future expansion tracks. A compliance & external-onboarding workstream (GST registration, Razorpay KYC, policies / Grievance Officer, DPDP, cosmetics labeling, vendor-feed confirmation) runs in parallel from Phase 0. Durations are indicative for an assistant-led build with the founder as product owner; they depend on review cycles and external onboarding. The Phase 0 foundation plus the Phase 1 schema make every later phase additive, not a rebuild.

## 7.1 Phase overview

| Phase | Objective | Indicative duration | Depends on |
|---|---|---|---|
| **0. Foundations** | **Environments, CI/CD, migrations, tests, observability, security, design system** | **1–2 weeks** | **—** |
| 1. Catalog + admin + ingestion | Catalog, admin, manual/spreadsheet ingestion, SSR + SEO, CMS-lite | 2–3 weeks | Phase 0 |
| 2. Customers + cart | Phone-OTP + guest checkout, addresses, cart/wishlist, reservation | 1–2 weeks | Phase 1 |
| 3. Checkout, payments & tax | Razorpay + COD, GST invoices, orders, refunds, notifications, coupons | 3–4 weeks | Phase 2; Razorpay KYC; GST |
| 4. Fulfillment + vendor automation | Routing, shipping + serviceability/COD/RTO, returns/RMA, automated ingestion | 3–4 weeks | Phase 3; vendor feeds |
| 5. Reviews, content & AI support | Reviews/UGC, content/blog/merchandising, chatbot + Jira | 2–3 weeks | Phase 1+ ; Jira |
| 6. Inventory & replenishment (ERP) | Replenishment engine, POs, FEFO, vendor scorecards | 2–3 weeks | Phase 4 |
| 7. Marketplace enablement (future) | Seller portal, buy box, commission & payouts | Future track | Phases 3–6 |

## 7.2 Phase detail

### Phase 0 — Foundations

- Separate dev / staging / production environments with isolated D1/R2/KV and secrets.
- CI/CD with preview deployments; versioned D1 schema migrations; unit + Playwright e2e test harness.
- Observability (logs, metrics, error tracking, alerts), backups/DR, and security hardening (WAF, rate limiting, Turnstile).
- Cloudflare Images; a base component/design system; data-access layer and outbox scaffold.
- Legal scaffolding: published policies, Grievance Officer, and consent capture.
- **Exit criteria:** code ships through CI to staging with tests passing; rollback and migrations are proven.

### Phase 1 — Catalog, admin & manual ingestion

- Seller-/fulfillment-/inventory-/tax-ready D1 schema incl. batch/expiry and money as minor units.
- Password-protected admin panel (Cloudflare Access) for categories, brands, products, variants, inventory; CMS-lite for banners/collections.
- Image upload to Cloudflare R2; manual entry and spreadsheet/CSV bulk upload through the ingestion pipeline (validate → review → publish).
- 15–20 dummy “Beyond Gorgeous” products; server-side-rendered product/category pages with structured data, sitemap, robots.txt.
- **Exit criteria:** founder can manage the catalog unaided; public pages render server-side with structured data.

### Phase 2 — Customers & cart

- Phone-OTP login and first-class guest checkout; profiles and saved addresses (pincode).
- Cart and wishlist; back-in-stock subscriptions; atomic stock reservation.
- **Exit criteria:** a guest can shop and a customer can log in via OTP and build a cart that holds reserved stock.

### Phase 3 — Checkout, payments & tax

- Razorpay (UPI/cards/net-banking/wallets) and cash-on-delivery; signed webhooks and idempotency keys.
- Order state machine and refunds; GST-compliant tax invoices; transactional notifications (email/SMS/WhatsApp); basic coupons; abandoned-cart recovery.
- **Exit criteria:** an end-to-end order can be paid (or placed as COD), invoiced, and confirmed with notifications.

### Phase 4 — Fulfillment & vendor automation

- Order-routing engine for dropship and self-fulfilled lines, including split shipments.
- Shipping via courier aggregator with pincode serviceability and COD/RTO handling; dropship tracking ingestion.
- Returns/RMA reverse logistics; automated ingestion (vendor API webhooks, scheduled pulls, email-in).
- **Exit criteria:** orders route automatically; a vendor feed updates stock without manual effort; returns flow end-to-end.

### Phase 5 — Reviews, content & AI support

- Reviews/UGC with moderation (and Review structured data); content/blog and merchandising (collections, bundles).
- Chatbot grounded in catalog, order data, and FAQ; Jira ticket escalation with context.
- **Exit criteria:** customers can review products; common queries are answered automatically; complex issues create a Jira ticket.

### Phase 6 — Inventory & replenishment (ERP)

- Stock ledger and derived on-hand/available; reorder policies with safety stock and reorder points.
- Replenishment engine with three modes per SKU: auto-PO, draft-for-approval, or alert-only.
- Purchase orders (sent via API or email), goods receipt, and low-stock alarms / Jira tickets.
- **Exit criteria:** a SKU falling below its reorder point automatically triggers its configured action.

### Phase 7 — Marketplace enablement (future)

- Seller onboarding & portal; multi-offer buy box; commission and automated payouts (Razorpay Route).
- Per-seller shipping, ratings, store pages, and GST/tax.
- **Exit criteria:** a third-party seller can list, sell, and be paid out — with no change to the core catalog schema.

## 7.3 Indicative milestones

Target dates are planning estimates from a 25 June 2026 start and will move with review cycles and external onboarding (Razorpay KYC, vendor feeds).

| Milestone | Marks completion of | Indicative target |
|---|---|---|
| **M1** | **Phase 1 — catalog & admin** | **Mid July 2026** |
| M2 | Phase 2 — customers & cart | End July 2026 |
| M3 | Phase 3 — payments live | Mid August 2026 |
| M4 | Phase 4 — fulfillment & automation | Mid September 2026 |
| M5 | Phase 5 — AI support | End September 2026 |
| Launch | Public launch (watermark removed) | October 2026 (target) |

## 8. Dependencies

| Dependency | Needed for | Owner / action |
|---|---|---|
| **Razorpay account + business KYC** | **Phase 3 payments** | **Founder — begin onboarding early** |
| Vendor data feeds (API / sheet / email) | Phase 4 automation | Founder — confirm each vendor's method |
| Courier aggregator account | Self-fulfilled shipping | Founder — sign up (e.g. Shiprocket) |
| Jira Cloud project | Phase 5 support | Founder — provision project |
| Claude API key | Phase 5 chatbot | Founder — provision and fund |
| Cloudflare D1 / R2 / Queues enabled | All phases | Build partner — enable in account |
| Product content (real) | Replacing dummy data | Founder — supply when available |
| GST registration & policies | Phase 3 tax + compliance | Founder — register; publish policies + Grievance Officer |
| Messaging providers (SMS/WhatsApp) | Phase 3 notifications | Founder — sign up (e.g. Gupshup/Twilio/Meta WABA) |
| Cosmetics labeling / CDSCO | Selling cosmetics legally | Founder — ensure label/import compliance |

## 9. Risk register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| **Vendors offer only spreadsheets/email, not APIs** | **High** | **Medium** | **Multi-source ingestion already designed; manual and file paths are first-class** |
| Razorpay KYC/onboarding delays go-live | Medium | High | Start KYC early, in parallel with Phase 1–2 |
| Overselling in hybrid inventory | Medium | High | Unified inventory + stock reservation at checkout |
| Vendor data quality is poor | Medium | Medium | Validation gates + human approval for new items |
| SEO ranking takes time to build | High | Medium | SSR + structured data from day one; set expectations |
| Next.js 16 Windows build issue recurs | Medium | Low | Build on Cloudflare Linux (already proven) |
| Scope creep delays launch | Medium | Medium | Phased roadmap; decision log; defer to V3 |
| Founder bandwidth / learning curve | Medium | Medium | Assistant-led build; incremental, reviewable steps |
| Running costs exceed expectations | Low | Medium | Cloudflare free tiers; monitor Claude API usage |
| Customer data/security incident | Low | High | Cloudflare Access, least privilege, cards via Razorpay |
| Non-compliance (GST / DPDP / e-commerce rules) | Medium | High | Compliance workstream from Phase 0; policies, Grievance Officer, GST invoices |
| COD return-to-origin (RTO) loss & fraud | High | Medium | Serviceability gating, COD limits/risk flags, RTO reconciliation |
| D1 outgrown by search / analytics | Medium | Medium | Data-access layer, dedicated search index, analytics off D1 |
| Shipping without engineering foundations | Medium | High | Phase 0: environments, CI/CD, migrations, tests, backups |

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Cosmetics expiry / recall handling | Low | High | Batch/lot + expiry (FEFO) in the ledger; recall by batch |

# 10. Assumptions & constraints

## 10.1 Assumptions

- Domains, DNS, SSL, and the .online redirect are live and stable (completed in V1).
- Vendors are whitelisted and willing to share price/stock/product data in some format.
- A business entity and bank account are available for Razorpay onboarding.
- Launch is India-first: INR pricing and UPI-friendly payments.
- The build is assistant-led, with the founder as product owner and reviewer.

## 10.2 Constraints

- The platform remains on the Cloudflare stack (Workers, D1, R2, KV, Queues).
- Documentation favours process clarity over code depth, reflecting the founder’s preference.
- Free/low-cost service tiers are preferred while volumes are low.

# 11. Decision log

Status legend: Decided, Default (recommended, not yet confirmed), Open.

| # | Decision | Status | Position |
|---|---|---|---|
| **1** | **Schema scope** | **Decided** | **Full seller/fulfillment/inventory/tax/compliance-ready from day one** |
| 2 | Variants | Decided | Full variants (shade/size with own SKU) |
| 3 | Product/listing separation | Decided | Cart/orders reference listings |
| 4 | Inventory model | Decided | Stock ledger + batch/expiry (FEFO) |
| 5 | Payments | Decided | Razorpay (+Route) and COD |
| 6 | Customer auth | Decided | Phone-OTP primary + guest checkout |
| 7 | Money representation | Decided | Integer minor units + currency |
| 8 | API design | Decided | API-first, versioned /v1 |
| 9 | Search | Decided | Dedicated index (not D1 LIKE) |
| 10 | Data access | Decided | Repository layer to avoid D1 lock-in |
| 11 | Reliability | Decided | Event/outbox pattern; idempotency keys |
| 12 | Tax | Decided | GST from day one |
| 13 | Replenishment default | Default | alert/approval at launch; auto_po opt-in |
| 14 | Shipping aggregator | Default | Shiprocket |
| 15 | Messaging providers | Open | TBD (e.g. Gupshup/Twilio/Meta WABA) |
| 16 | Per-vendor data method | Open | Confirm API vs spreadsheet/email per vendor |
| 17 | Self vs dropship at launch | Open | Likely all dropship initially |
| 18 | Marketplace go/no-go & timing | Open | Design now; build (Phase 7) only if needed |
| 19 | Recommendations engine | Future | Later (AI) |

## 12. Glossary

| Term | Meaning |
|---|---|
| **Modular monolith** | **One application divided into clean internal modules, not separate services** |
| PIM | Product Information Management — the central hub that owns and distributes product data |
| Hub-and-spoke | Many sources feed one central hub that normalises and distributes data |
| Ingestion pipeline | The standard path every feed follows: receive, normalise, validate, map, approve, publish |
| Adapter / connector | A small translator converting one source's format into our standard format |
| Field mapping | Config saying "their column = our field," so onboarding a feed needs no new code |
| Idempotent upsert | Insert-or-update that is safe to repeat; duplicates cause no harm |
| Dropshipping | The vendor ships directly to the customer; we hold no stock |
| Self-fulfilled | We hold inventory and ship it ourselves via a shipping partner |
| Courier aggregator | One integration giving access to many courier companies |
| Order routing | Deciding which location or source fulfils each item of an order |
| Stock reservation | Holding stock for an in-progress order to prevent overselling |
| SSR | Server-side rendering — the server sends complete HTML for SEO and speed |
| JSON-LD | Structured data in a page enabling Google rich results (price, stars, breadcrumbs) |
| Canonical URL | The one official URL for a page, told to Google to avoid duplicate content |
| Slug | The human-readable part of a URL |
| Seller / merchant | An entity that lists and sells products and receives payment; one house seller at launch |

| Term | Meaning |
| --- | --- |
| Listing / offer | A seller’s sellable unit (price, stock, fulfilment) for a variant; cart and orders reference listings |
| Buy box | Logic that picks the default offer when multiple sellers list the same item (future) |
| Split settlement | Paying multiple sellers from one customer payment (e.g. Razorpay Route) |
| Stock ledger | Immutable record of inventory movements; on-hand is derived from it |
| On-hand / reserved / available | Physically held / allocated to orders / sellable now |
| On-order (incoming) | Quantity inbound on open purchase orders |
| Safety stock | Buffer stock held to absorb demand and lead-time variability |
| Reorder point (ROP) | Stock level at which replenishment is triggered |
| Reorder qty / EOQ | How much to reorder; EOQ is the cost-optimal lot size |
| Lead time | Time from placing a purchase order to stock being available |
| Purchase order (PO) | An order we place on a vendor to replenish stock |
| Goods receipt (GRN) | Recording stock received against a PO; posts ledger entries |
| Replenishment | Restocking via auto-PO, approval, or alert when stock is low |
| Guest checkout / OTP | Buying without an account / one-time-password phone login |
| COD / RTO / RMA | Cash on delivery / return-to-origin / return merchandise authorisation |
| FEFO / batch (lot) | First-expiry-first-out / a dated production run |
| Serviceability | Whether a pincode can be delivered to (and supports COD) |
| HSN / GSTIN / e-invoice | Tax code / GST registration number / govt-validated GST invoice |
| DPDP / Grievance Officer | India data-protection law / mandated complaint contact |
| Tokenization | Replacing card numbers with tokens (RBI rule; cards never stored) |

| Term | Meaning |
| --- | --- |
| Outbox pattern | Persist events then dispatch, so side-effects are never lost |
| Data-access layer | Repository abstraction over the database to avoid lock-in |
| WAF / Turnstile / SLO | Web firewall / bot challenge / service-level objective |
| Minor units | Money stored as integers (paise), never floats |

**Canonical technical reference:** docs/ARCHITECTURE.md in the project repository, version-controlled in Git. This Word document and that file are kept aligned by version number.